

DSA Programs 12–25

12. Stack - Infix to Postfix

Input: A+B*C Output: ABC*+

```
#include<stdio.h>
```

```
#include<ctype.h>
```

```
char stack[100];
```

```
int top=-1;
```

```
void push(char x){
```

```
    stack[++top]=x;
```

```
}
```

```
char pop(){
```

```
    return stack[top--];
```

```
}
```

```
int priority(char x){
```

```
    if(x=='+'||x=='-') return 1;
```

```
    if(x=='*'||x=='/') return 2;
```

```
    return 0;
```

```
}
```

```
int main(){
```

```
    char exp[100], *e, x;
```

```
    scanf("%s", exp);
```

```
e=exp;
```

```
while(*e){
```

```
    if(isalnum(*e))
```

```
        printf("%c", *e);
```

```
    else if(*e=='(')
```

```
        push(*e);
```

```
    else if(*e==')'){
```

```
        while((x=pop())!='(')
```

```
            printf("%c", x);
```

```
    }
```

```
    else{
```

```
        while(top!=-1 && priority(stack[top])>=priority(*e))
```

```
            printf("%c", pop());
```

```
        push(*e);
```

```
    }
```

```
    e++;
```

```
}
```

```
while(top!=-1)
```

```
    printf("%c", pop());
```

```
return 0;
```

```
}
```

13. Queue using Array

Input: 10,20,30 Output: Deleted 10

```
#include<stdio.h>
```

```
int q[100], f=-1, r=-1;
```

```
void enqueue(int x){
```

```
    if(r==99) return;
```

```
    if(f==-1)
```

```
        f=0;
```

```
    q[++r]=x;
```

```
}
```

```
void dequeue(){
```

```
    if(f==-1 || f>r) return;
```

```
    printf("Deleted %d", q[f++]);
```

```
}
```

```
int main(){
```

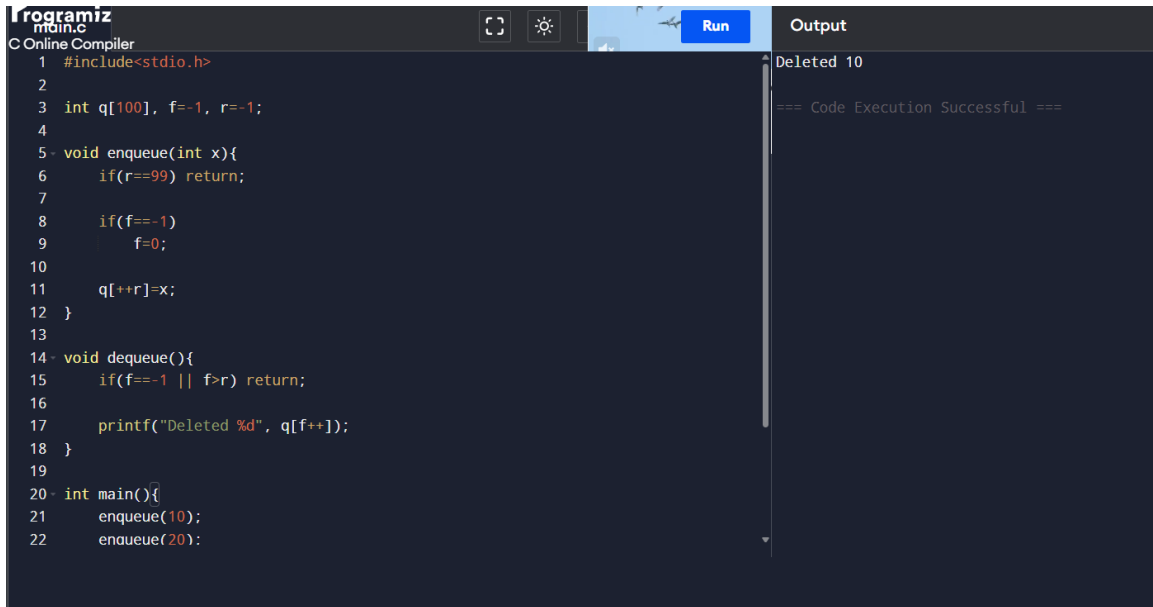
```
    enqueue(10);
```

```
    enqueue(20);
```

```
    enqueue(30);
```

```
    dequeue();
```

```
return 0;
```

A screenshot of an online C compiler interface. The main window shows a C program with the following code:

```
1 #include<stdio.h>
2
3 int q[100], f=-1, r=-1;
4
5 void enqueue(int x){
6     if(r==99) return;
7
8     if(f==1)
9         f=0;
10
11     q[++r]=x;
12 }
13
14 void dequeue(){
15     if(f==1 || f>r) return;
16
17     printf("Deleted %d", q[f++]);
18 }
19
20 int main(){
21     enqueue(10);
22     enqueue(20);
```

The interface includes a 'Run' button and an 'Output' panel on the right. The output panel displays the text 'Deleted 10' and '=== Code Execution Successful ==='. The compiler is identified as 'Programiz Online Compiler'.

```
}
```

14. Tree Traversal

Input: Binary Tree Output: Inorder sequence

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
```

```
    int data;
```

```
    struct node *l,*r;
```

```
};
```

```
struct node* newNode(int x){
```

```
    struct node* n=(struct node*)malloc(sizeof(struct node));
```

```
    n->data=x;
```

```
    n->l=n->r=NULL;
```

```
    return n;
```

```
}
```

```

void inorder(struct node* t){

    if(t){

        inorder(t->l);

        printf("%d ", t->data);

        inorder(t->r);

    }

}

int main(){

    struct node* root=newNode(1);

    root->l=newNode(2);

    root->r=newNode(3);

    inorder(root);

    return 0;

}

```

```

main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3
4  struct node{
5      int data;
6      struct node *l,*r;
7  };
8
9  struct node* newNode(int x){
10     struct node* n=(struct node*)malloc(sizeof(struct node));
11     n->data=x;
12     n->l=n->r=NULL;
13     return n;
14 }
15
16 void inorder(struct node* t){
17     if(t){
18         inorder(t->l);
19         printf("%d ", t->data);
20         inorder(t->r);
21     }
22 }

```

2 1 3

=== Code Execution Successful ===

15. Hashing Linear Probing

Input: 23,43,13 Output: Hash Table

```
#include<stdio.h>
```

```
int ht[10];
```

```
int main(){
```

```
    int keys[]={23,43,13};
```

```
    for(int i=0;i<10;i++)
```

```
        ht[i]=-1;
```

```
    for(int i=0;i<3;i++){
```

```
        int h=keys[i]%10;
```

```
        while(ht[h]!=-1)
```

```
            h=(h+1)%10;
```

```
        ht[h]=keys[i];
```

```
    }
```

```
    for(int i=0;i<10;i++)
```

```
        printf("%d -> %d\n", i, ht[i]);
```

```
    return 0;
```

```

}
}

Programiz
main.c
C Online Compiler
3 int ht[10];
4
5 int main(){
6     int keys[]={23,43,13};
7
8     for(int i=0;i<10;i++)
9         ht[i]=-1;
10
11     for(int i=0;i<3;i++){
12         int h=keys[i]%10;
13
14         while(ht[h]!=-1)
15             h=(h+1)%10;
16
17         ht[h]=keys[i];
18     }
19
20     for(int i=0;i<10;i++)
21         printf("%d -> %d\n", i, ht[i]);
22
23     return 0;
24 }

```

Output

```

0 -> -1
1 -> -1
2 -> -1
3 -> 23
4 -> 43
5 -> 13
6 -> -1
7 -> -1
8 -> -1
9 -> -1

=== Code Execution Successful ===

```

16. Insertion Sort

Input: 5 2 4 Output: 2 4 5

```
#include<stdio.h>
```

```
int main(){
```

```
    int a[5]={5,2,4}, n=3, i, j, key;
```

```
    for(i=1;i<n;i++){
```

```
        key=a[i];
```

```
        j=i-1;
```

```
        while(j>=0 && a[j]>key){
```

```
            a[j+1]=a[j];
```

```
            j--;
```

```
        }
```

```
        a[j+1]=key;
```

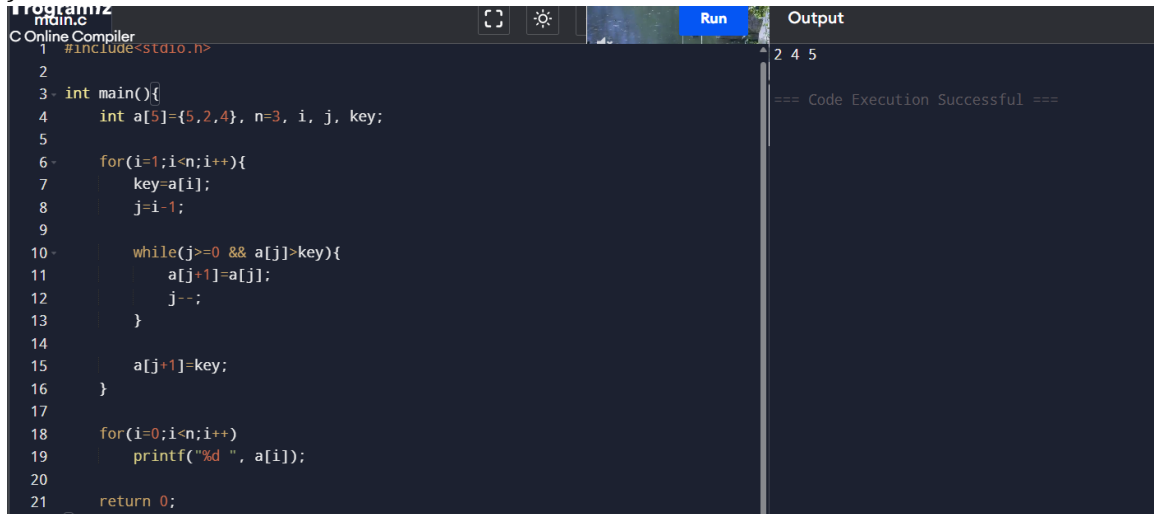
```
}
```

```
for(i=0;i<n;i++)
```

```
    printf("%d ", a[i]);
```

```
return 0;
```

```
}
```



```
Programiz
Online Compiler
main.c
1 #include<stdio.h>
2
3 int main(){
4     int a[5]={5,2,4}, n=3, i, j, key;
5
6     for(i=1;i<n;i++){
7         key=a[i];
8         j=i-1;
9
10        while(j>=0 && a[j]>key){
11            a[j+1]=a[j];
12            j--;
13        }
14
15        a[j+1]=key;
16    }
17
18    for(i=0;i<n;i++)
19        printf("%d ", a[i]);
20
21    return 0;
```

Output

```
2 4 5
=== Code Execution Successful ===
```

17. Priority Queue

Input: 10,20 Output: Highest priority element

```
#include<stdio.h>
```

```
int main(){
```

```
    int a[5]={10,20}, n=2, max=0;
```

```
    for(int i=1;i<n;i++)
```

```
        if(a[i]>a[max])
```

```
            max=i;
```



```
printf("Highest=%d", a[max]);
```

```
return 0;
```

```
}
```

main.c	Output
<pre>1 #include<stdio.h> 2 3 int main(){ 4 int a[5]={10,20}, n=2, max=0; 5 6 for(int i=1;i<n;i++) 7 if(a[i]>a[max]) 8 max=i; 9 10 printf("Highest=%d", a[max]); 11 12 return 0; 13 }</pre>	<pre>Highest=20 === Code Execution Successful ===</pre>

18. Merge Sort

Input: 3 1 Output: 1 3

```
#include<stdio.h>
```

```
void merge(int a[], int l, int m, int r){
```

```
    int i=l, j=m+1, k=0, b[100];
```

```
    while(i<=m && j<=r)
```

```
        b[k++]=(a[i]<a[j])?a[i++]:a[j++];
```

```
    while(i<=m)
```

```
        b[k++]=a[i++];
```

```
    while(j<=r)
```

```
        b[k++]=a[j++];
```

```
    for(i=l,k=0;i<=r;i++,k++)  
        a[i]=b[k];  
}
```

```
void ms(int a[], int l, int r){  
    if(l<r){  
        int m=(l+r)/2;  
  
        ms(a,l,m);  
        ms(a,m+1,r);  
        merge(a,l,m,r);  
    }  
}
```

```
int main(){  
    int a[]={3,1};  
  
    ms(a,0,1);  
  
    printf("%d %d", a[0], a[1]);  
  
    return 0;
```

```

}
Programiz
main.c
C Online Compiler
1 #include<stdio.h>
2
3 void merge(int a[], int l, int m, int r){
4     int i=l, j=m+1, k=0, b[100];
5
6     while(i<=m && j<=r)
7         b[k++]=(a[i]<a[j])?a[i++]:a[j++];
8
9     while(i<=m)
10        b[k++] = a[i++];
11
12    while(j<=r)
13        b[k++] = a[j++];
14
15    for(i=l, k=0; i<=r; i++, k++)
16        a[i] = b[k];
17 }
18
19 void ms(int a[], int l, int r){
20     if(l<r){
21         int m=(l+r)/2;

```

19. Quick Sort

Input: 3 1 Output: 1 3

```
#include<stdio.h>
```

```

void qs(int a[], int l, int r){

    if(l<r){

        int p=a[l], i=l, j=r, temp;

        while(i<j){

            while(a[i]<=p && i<r)

                i++;

            while(a[j]>p)

                j--;

            if(i<j){

                temp=a[i];

```

```
        a[i]=a[j];  
        a[j]=temp;  
    }  
}
```

```
temp=a[l];  
a[l]=a[j];  
a[j]=temp;
```

```
    qs(a,l,j-1);  
    qs(a,j+1,r);  
}  
}
```

```
int main(){  
    int a[]={3,1};  
  
    qs(a,0,1);  
  
    printf("%d %d", a[0], a[1]);  
  
    return 0;
```

```

}
1 #include<stdio.h>
2
3 void qs(int a[], int l, int r){
4     if(l<r){
5         int p=a[l], i=l, j=r, temp;
6
7         while(i<j){
8             while(a[i]<=p && i<r)
9                 i++;
10
11             while(a[j]>p)
12                 j--;
13
14             if(i<j){
15                 temp=a[i];
16                 a[i]=a[j];
17                 a[j]=temp;
18             }
19         }
20
21     }
22 }
23
24 int main(){
25     int a[]={4,1,3};
26     qs(a,0,2);
27     printf("Sorted array: ");
28     for(int i=0; i<3; i++)
29         printf("%d ", a[i]);
30     printf("\n");
31     return 0;
32 }

```

1 3
=== Code Execution Successful ===

20. Heap Sort

Input: 4 1 3 Output: 1 3 4

```
#include<stdio.h>
```

```
void heapify(int a[], int n, int i){
```

```
    int largest=i;
```

```
    int l=2*i+1;
```

```
    int r=2*i+2;
```

```
    int temp;
```

```
    if(l<n && a[l]>a[largest])
```

```
        largest=l;
```

```
    if(r<n && a[r]>a[largest])
```

```
        largest=r;
```

```
    if(largest!=i){
```

```
        temp=a[i];
```

```
        a[i]=a[largest];
```

```
    a[largest]=temp;
```

```
    heapify(a,n,largest);
```

```
}
```

```
}
```

```
int main(){
```

```
    int a[]={4,1,3};
```

```
    int n=3;
```

```
    int temp;
```

```
    for(int i=n/2-1;i>=0;i--)
```

```
        heapify(a,n,i);
```

```
    for(int i=n-1;i>0;i--){
```

```
        temp=a[0];
```

```
        a[0]=a[i];
```

```
        a[i]=temp;
```

```
        heapify(a,i,0);
```

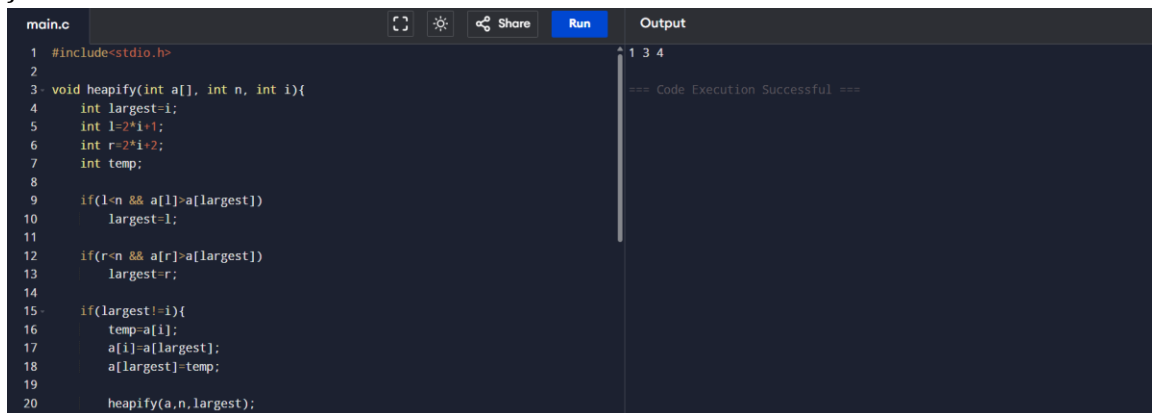
```
}
```

```
    for(int i=0;i<n;i++)
```

```
        printf("%d ",a[i]);
```

```
    return 0;
```

}



The screenshot shows a code editor with a file named 'main.c'. The code implements a heapify function for a binary tree. The function takes an array 'a', its size 'n', and an index 'i'. It finds the largest element among 'i' and its children, and swaps it with 'i' if necessary. The output window shows the numbers '1 3 4' and a message '=== Code Execution Successful ==='.

```
main.c
1 #include<stdio.h>
2
3 void heapify(int a[], int n, int i){
4     int largest=i;
5     int l=2*i+1;
6     int r=2*i+2;
7     int temp;
8
9     if(l<n && a[l]>a[largest])
10         largest=l;
11
12     if(r<n && a[r]>a[largest])
13         largest=r;
14
15     if(largest!=i){
16         temp=a[i];
17         a[i]=a[largest];
18         a[largest]=temp;
19
20         heapify(a,n,largest);
21     }
```

1 3 4
=== Code Execution Successful ===

21. AVL Tree (Insertion)

Input: 10 20 30 Output: Balanced Tree

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int key,height;
    struct node *l,*r;
};
```

```
int h(struct node *n){
    return n ? n->height : 0;
}
```

```
int max(int a,int b){
    return a>b?a:b;
}
```

```

struct node* newNode(int k){
    struct node *n=(struct node*)malloc(sizeof(struct node));
    n->key=k;
    n->l=n->r=NULL;
    n->height=1;
    return n;
}

```

```

struct node* rr(struct node *y){
    struct node *x=y->l;
    y->l=x->r;
    x->r=y;

    y->height=max(h(y->l),h(y->r))+1;
    x->height=max(h(x->l),h(x->r))+1;

    return x;
}

```

```

struct node* lr(struct node *x){
    struct node *y=x->r;
    x->r=y->l;
    y->l=x;

    x->height=max(h(x->l),h(x->r))+1;
    y->height=max(h(y->l),h(y->r))+1;
}

```



```
    return y;
}
```

```
int bf(struct node *n){
    return h(n->l)-h(n->r);
}
```

```
struct node* insert(struct node *n,int k){
    if(!n)
        return newNode(k);
```

```
    if(k<n->key)
        n->l=insert(n->l,k);
    else
        n->r=insert(n->r,k);
```

```
    n->height=1+max(h(n->l),h(n->r));
```

```
    int b=bf(n);
```

```
    if(b>1 && k<n->l->key)
        return rr(n);
```

```
    if(b<-1 && k>n->r->key)
        return lr(n);
```

```

        return n;
    }

int main(){

    struct node *root=NULL;


    root=insert(root,10);

    root=insert(root,20);

    root=insert(root,30);


    printf("Balanced Tree Created");

    return 0;
}

```

The screenshot shows a code editor with a file named 'main.c'. The code implements a balanced tree structure with functions for height calculation, maximum value, and node creation. The output window on the right shows the message 'Balanced Tree Created' and '=== Code Execution Successful ==='.

```

main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3
4  struct node{
5      int key,height;
6      struct node *l,*r;
7  };
8
9  int h(struct node *n){
10     return n ? n->height : 0;
11 }
12
13 int max(int a,int b){
14     return a>b?a:b;
15 }
16
17 struct node* newNode(int k){
18     struct node *n=(struct node*)malloc(sizeof(struct node));
19     n->key=k;
20     n->l=n->r=NULL;
21     n->height=1;

```

Output

```

Balanced Tree Created

=== Code Execution Successful ===

```

22. Dijkstra Algorithm

Input: Graph Output: Shortest Path

```
#include<stdio.h>
```

```
#define V 3
```

```
int min(int d[], int v[]){
```

```
    int m=999, mi;
```

```
    for(int i=0;i<V;i++){
```

```
        if(!v[i] && d[i]<=m){
```

```
            m=d[i];
```

```
            mi=i;
```

```
        }
```

```
    return mi;
```

```
}
```

```
void dij(int g[V][V], int s){
```

```
    int d[V], v[V]={0};
```

```
    for(int i=0;i<V;i++){
```

```
        d[i]=999;
```

```
    d[s]=0;
```

```

for(int c=0;c<V-1;c++){
    int u=min(d,v);
    v[u]=1;

    for(int i=0;i<V;i++)
        if(!v[i] && g[u][i] &&
            d[u]+g[u][i]<d[i])
            d[i]=d[u]+g[u][i];
    }

    for(int i=0;i<V;i++)
        printf("%d ",d[i]);
}

int main(){
    int g[V][V]={{0,1,4},
                  {1,0,2},
                  {4,2,0}};

    dij(g,0);

    return 0;
}

```

```
main.c 0 1 3
1 #include<stdio.h>
2
3 #define V 3
4
5 int min(int d[], int v[]){
6     int m=999, mi;
7
8     for(int i=0;i<V;i++){
9         if(!v[i] && d[i]<=m){
10             m=d[i];
11             mi=i;
12         }
13     }
14     return mi;
15 }
16
17 void dij(int g[V][V], int s){
18     int d[V], v[V]={0};
19
20     for(int i=0;i<V;i++){
```

=== Code Execution Successful ===

23. Prim's Algorithm

Input: Graph Output: MST

```
#include<stdio.h>
```

```
#define V 3
```

```
int main(){
```

```
    int g[V][V]={0,1,4},
        {1,0,2},
        {4,2,0}};
```

```
    int selected[V]={0};
```

```
    int x,y;
```

```
    selected[0]=1;
```

```
    for(int e=0;e<V-1;e++){
```

```
        int min=999;
```

```

for(int i=0;i<V;i++)

    if(selected[i])

        for(int j=0;j<V;j++)

            if(!selected[j] && g[i][j]){

                if(min>g[i][j]){

                    min=g[i][j];

                    x=i;

                    y=j;

                }

            }

        printf("%d-%d\n",x,y);

        selected[y]=1;

    }

return 0;

}

```

```

main.c
13 selected[0]=1;
14
15 for(int e=0;e<V-1;e++){
16     int min=999;
17
18     for(int i=0;i<V;i++){
19         if(selected[i])
20             for(int j=0;j<V;j++){
21                 if(!selected[j] && g[i][j]){
22                     if(min>g[i][j]){
23                         min=g[i][j];
24                         x=i;
25                         y=j;

```

Output

```

0-1
1-2

=== Code Execution Successful ===

```

24. Kruskal Algorithm

Input: Edges Output: MST

```
#include<stdio.h>
```

```
int parent[10];
```

```
int find(int i){
```

```
    while(parent[i])
```

```
        i=parent[i];
```

```
    return i;
```

```
}
```

```
void uni(int i,int j){
```

```
    parent[j]=i;
```

```
}
```

```
int main(){
```

```
    int cost[3][3]={0,1,4},
```

```
                {1,0,2},
```

```
                {4,2,0}};
```

```
    int mincost=0;
```

```
    for(int i=0;i<3;i++)
```

```
        parent[i]=0;
```

```
for(int e=0;e<2;e++){  
    int min=999,a,b;  
  
    for(int i=0;i<3;i++){  
        for(int j=0;j<3;j++){  
            if(cost[i][j]<min && cost[i][j]!=0){  
                min=cost[i][j];  
                a=i;  
                b=j;  
            }  
        }  
    }  
  
    int u=find(a);  
    int v=find(b);  
  
    if(u!=v){  
        uni(u,v);  
        printf("%d-%d\n",a,b);  
        mincost+=min;  
    }  
  
    cost[a][b]=cost[b][a]=999;  
}  
  
printf("Cost=%d",mincost);
```



```

return 0;
}

```

```

main.c
1 #include<stdio.h>
2
3 int parent[10];
4
5 int find(int i){
6     while(parent[i])
7         i=parent[i];
8
9     return i;
10 }
11
12 void uni(int i,int j){
13     parent[j]=i;
14 }
15
16 int main(){
17     int cost[3][3]={0,1,4},
18                 {1,0,2},
19                 {4,2,0}};
20
21     int mincost=0;
22

```

```

Output
0-1
1-2
Cost=3

=== Code Execution Successful ===

```

25. Hashing using linear probing

```
#include <stdio.h>
```

```
#define SIZE 10
```

```
int main() {
```

```
    int hashTable[SIZE];
```

```
    int keys[] = {23, 43, 13, 27, 33};
```

```
    int n = 5;
```

```
    for(int i = 0; i < SIZE; i++)
```

```
        hashTable[i] = -1;
```

```
    for(int i = 0; i < n; i++) {
```

```
        int key = keys[i];
```

```
int index = key % SIZE;
```

```
while(hashTable[index] != -1)
```

```
    index = (index + 1) % SIZE;
```

```
    hashTable[index] = key;
```

```
}
```

```
printf("Hash Table:\n");
```

```
for(int i = 0; i < SIZE; i++) {
```

```
    if(hashTable[i] != -1)
```

```
        printf("%d --> %d\n", i, hashTable[i]);
```

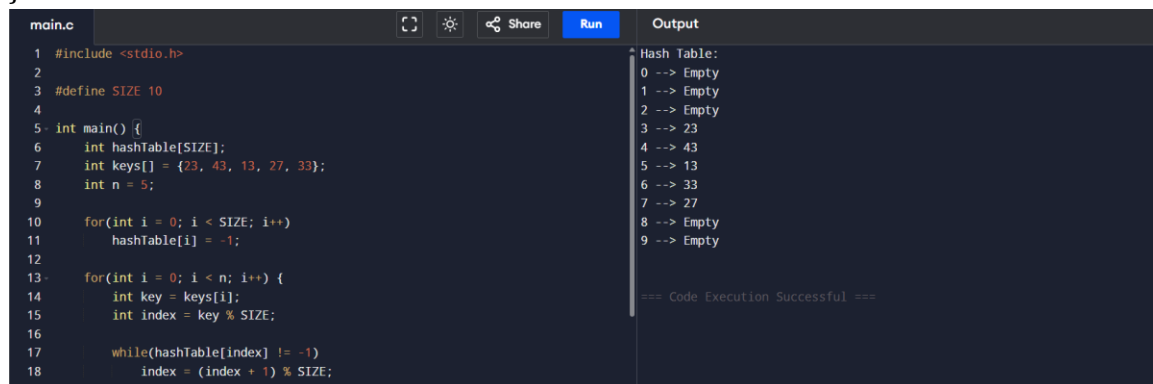
```
    else
```

```
        printf("%d --> Empty\n", i);
```

```
}
```

```
return 0;
```

```
}
```



The screenshot shows a C code editor with a file named `main.c`. The code implements a hash table with a size of 10. It initializes the table with -1 for all slots. Then, it inserts five keys: 23, 43, 13, 27, and 33. The insertion process uses a linear probing algorithm. The output window shows the resulting hash table:

```
Hash Table:
0 --> Empty
1 --> Empty
2 --> Empty
3 --> 23
4 --> 43
5 --> 13
6 --> 33
7 --> 27
8 --> Empty
9 --> Empty
```

The code execution was successful.